



東華大學
DONGHUA UNIVERSITY

神经网络与深度学习

DATA

计算机科学与技术学院

主讲教师：宋晖

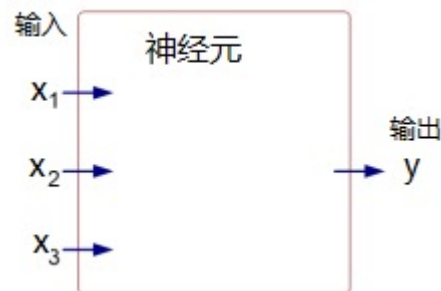
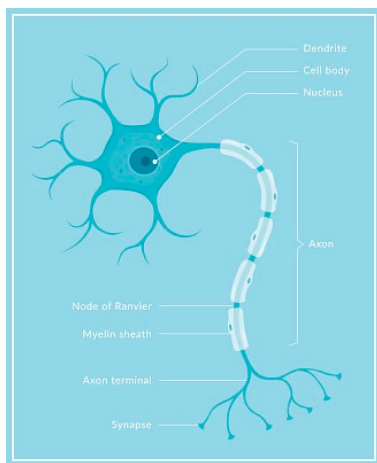
6.1 神经网络和深度学习

- 神经网络，也称为人工神经网络（Artificial Neural Network, ANN）
 - 20世纪80 年代开始研究，后陷入低潮
 - 随着计算能力增强和大数据出现，深度学习（也就是深度神经网络）技术呈爆发式发展
 - 目前是机器学习以及人工智能领域最重要的方法之一



6.1.1 神经元与感知器

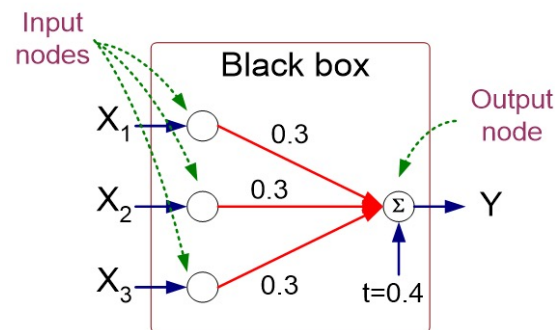
- 神经网络模拟人脑的神经网络来处理问题
 - 人脑思维基础是神经元，神经元相互连接
 - 当某个神经元接受输入，达到某种状态，它就会“兴奋”，向相连神经元发送化学物质
 - “人造神经元”模型，称为**感知器**（perceptron）



感知器计算模型

- 简化模型，约定输入和输出只有：1 或 0
 - 感知器模型由输入结点、输出结点和权重连接线组成
 - 输出结点将输入结点值乘以权重后加起来，然后和一个阈值 t 比较，决定输出1或0

X_1	X_2	X_3	Y
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	0
0	1	0	0
0	1	1	1
0	0	0	0



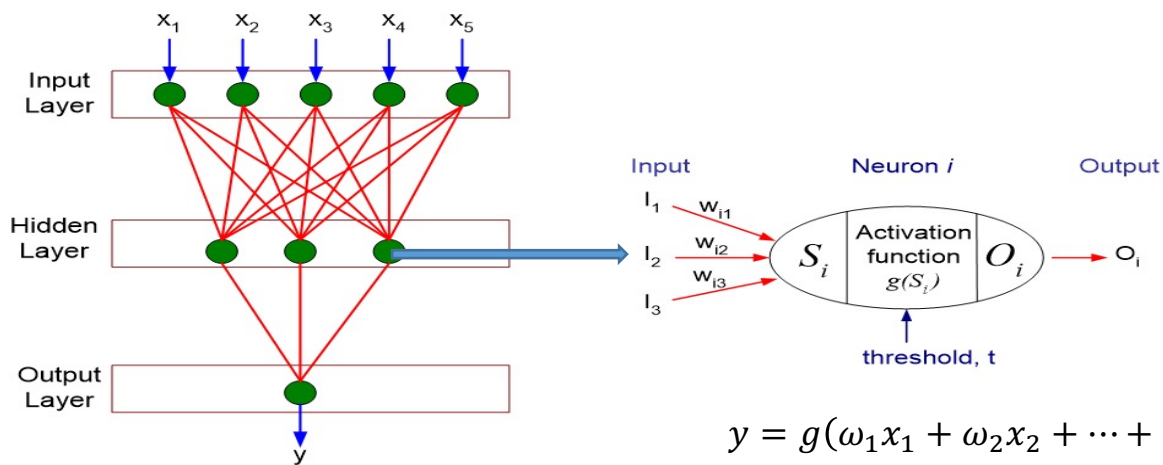
$$y = I(0.3x_1 + 0.3x_2 + \dots + 0.3x_3 - 0.4)$$

$I(z)$ 激活函数
(activation function)

其中：
$$I(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & z \leq 0 \end{cases}$$

神经网络模型

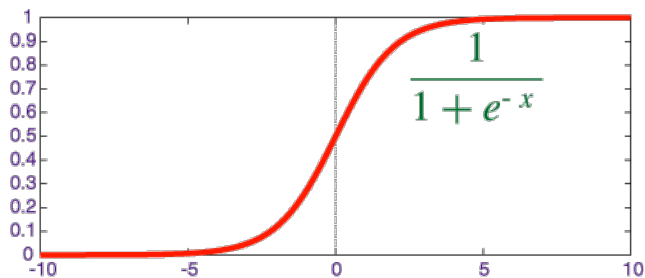
- 单个感知器能够处理线性可分问题
- 线性不可分问题，需考虑使用多层神经元
 - 输入层与输出层之间的神经元被称为隐藏层（hidden layer）



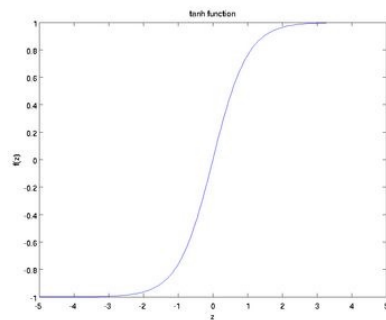
激活函数

- 常用sigmoid、tanh（双曲正切）、ReLU等函数

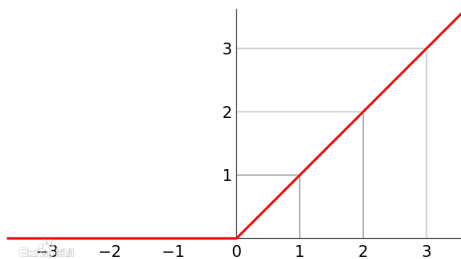
sigmoid $g(x) = \frac{1}{1+e^{-x}}$



tanh $g(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



ReLU $g(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$

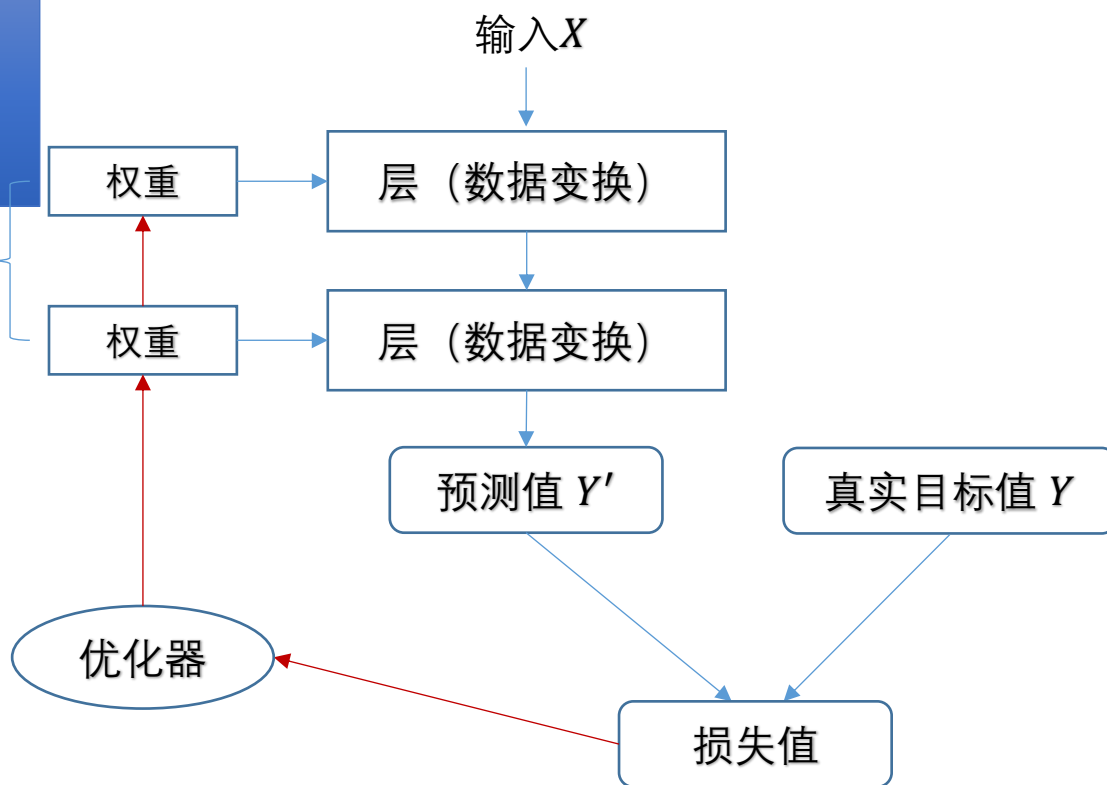


神经网络训练方法

- 基于已知分析结果的数据集，学习神经网络各层的参数

- 开始网络的权重随机赋值，随着训练过程循环足够多的次数，得到的权重使损失函数最小

模型的参数



- 调节由优化器 (optimizer) 实现，即反向传播 (BP, Error BackPropagation) 算法

- 将损失值作为反馈信号，对权重值进行微调，以降低损失值

- 设定损失/目标函数 (loss/object function)

分类/回归神经网络

- 分类

- 二分类问题，输出层只需要1个结点
 - 输出层使用sigmoid激活函数
 - 损失函数，采用binary_crossentropy
- 多分类问题，输出层需要多个输出结点，每个结点对应一种类型，输出值表示属于该类型的概率
 - 输出层使用softmax激活函数
 - 损失函数，采用categorical_crossentropy

- 回归

- 预测连续值，而不是离散的标签，如:明天的天气温度，地区房屋价格等
- 神经网络的输出层只有一个结点，不需要激活函数
- 损失函数，使用线性回归的目标函数，均方误差MSE

神经网络分类实现

- Scikit-learn的神经网络实现：MLPClassifier类

模型初始化：**mlp = MLPClassifier(solver,activation,hidden_layer_sizes,
alpha,max_iter,random_state,...)**

参数说明：	
solver	优化权重的算法：{'lbfgs', 'sgd', 'adam'}, 默认adam
activation	激活函数，取值{'identity', 'logistic', 'tanh', 'relu'}, 默认relu
hidden_layer_sizes	神经网络结构，用元祖表示，其中元祖第n个元素值表示，第n层的神经元个数。例：(5,10,5)，表示3隐层，每层的结点数分别为5、10和5
alpha	正则化惩罚项参数，缺省0.0001
max_iter	最大迭代次数，神经网络参数的BP学习算法的学习次数
random_state	随机数种子

例6-1：神经网络分类（1）

- 使用神经网络对鸢尾花数据集进行分类分析

1) 读取数据，统计样本特征

```
filename = 'data\iris.data'
data = pd.read_csv(filename, header = None)
data.columns = ['sepal length', 'sepal width', 'petal length', 'petal width', 'class']
data.iloc[0:5, :]
print( data['class'].value_counts() )
data.groupby('class').mean()
```

每类花样本数

```
Iris-setosa      50
Iris-virginica   50
Iris-versicolor  50
Name: class, dtype: int64
```

每类花各特征的均值

	sepal length	sepal width	petal length	petal width
class				
Iris-setosa	5.006	3.418	1.464	0.244
Iris-versicolor	5.936	2.770	4.260	1.326
Iris-virginica	6.588	2.974	5.552	2.026

每类花各特征的方差

	sepal length	sepal width	petal length	petal width
class				
Iris-setosa	0.124249	0.145180	0.030106	0.011494
Iris-versicolor	0.266433	0.098469	0.220816	0.039106
Iris-virginica	0.404343	0.104004	0.304588	0.075433

例6-1：神经网络分类（2）

2) 数据预处理，类别使用整数表示

```
data.loc[ data['class'] == 'Iris-setosa', 'class' ] = 0
data.loc[ data['class'] == 'Iris-versicolor', 'class' ] = 1
data.loc[ data['class'] == 'Iris-virginica', 'class' ] = 2
X = data.iloc[:,0:4]
y = data.iloc[:,4]
```

3) 创建神经网络分类器，训练网络结点连接权重以及偏差

```
from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(solver='lbfgs',alpha=1e-5,hidden_layer_sizes=(5, 5),
                    random_state=1)
mlp.fit(X,y)
mlp.score(X,y)
```

2个隐藏层，
每层5个结点

训练数据集上预测正确率达到98.6%

4) 分类器性能评估

```
from sklearn import metrics
y_predicted = mlp.predict(X_test)
print("Classification report for %s" % clf)

print (metrics.classification_report(y_test, y_predicted) )
print( "Confusion matrix:\n", metrics.confusion_matrix(y_test, y
_predicted) )
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	50
1	0.98	0.98	0.98	50
2	0.98	0.98	0.98	50
avg / total	0.99	0.99	0.99	150

Confusion matrix:
[[50 0 0]
[0 49 1]
[0 1 49]]

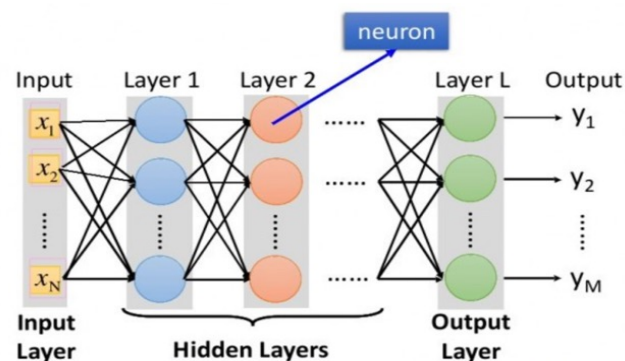
思考与练习

1. 调整MLP分类器的参数solver，比较不同参数的模型在鸢尾花数据集上的分类性能。
2. 在MLP训练函数fit()前后增加计时功能，设置不同隐层数目，比较训练所耗费的时间，以及模型分类的准确性。MLP模型是否结点越多分类性能越好？

【提示：】 计时函数：`import time`

6.2 深度学习

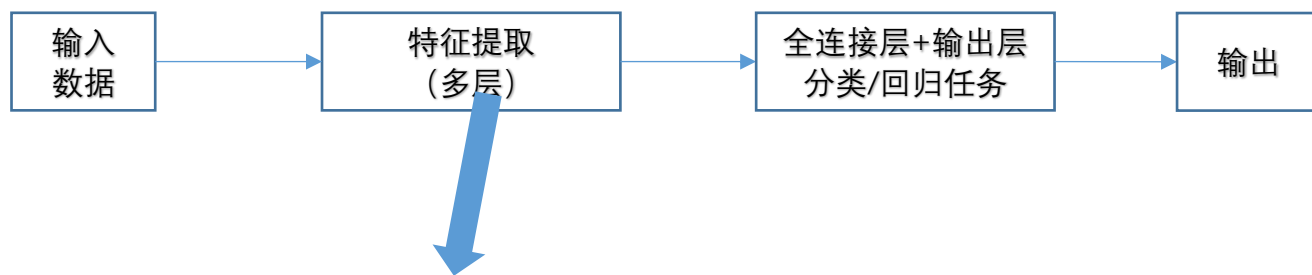
- 神经网络训练速度慢，学习算法存在梯度消失的问题
- 2006年，加拿大科学家Hinton与合作者发表了深度学习（Deep Learning）论文
 - 借助统计力学里“玻尔兹曼分布”的概念，改造了神经网络的学习机制
- 深度学习基于深度神经网络实现
 - 具有很多隐藏层（超过一层）、每个隐藏层具有很多结点的神经网络
- 其他机器学习方法，也被称为浅层学习（shallow learning）
 - 仅学习一两层的数据表示



深度神经网络结构

- 深度学习基本思想

- 从输入的数据中进行预先训练，以发现数据自身的重要特征
- 根据提取的特征建立初始化的神经网络，再基于分类/回归等标签进行学习，对网络参数进行微调



- 处理不同领域数据，研究适应性的网络结构
 - 图像、语音：卷积神经网络（CNN）
 - 文本、序列数据：递归神经网络（RNN）、长短期记忆网络（LSTM）、Transformer（Bert/XLNet）等

数据已经能够较好地表示研究对象的特征，如已结构化的数据集，小数据集等，应采用浅层机器学习模型

深度神经网络框架Keras

- Python的深度学习框架
 - 以Tensorflow、CNTK、或Theano为计算后台
 - 支持现有的常见模型CNN、RNN、LSTM等
 - 高度模块化，易于学习和使用
- Keras支持模型化编程
 - 序贯（Sequential）模型
 - 简单的线性模型，由多个神经网络层按输入输出顺序线性堆叠而成
 - 只具有一个输出
 - 函数式（Functional）模型
 - 在序贯模型基础上，允许用户定义多输出、非循环有向或具有共享层的结构

深度学习库Keras

- 在Anaconda集成环境中安装Keras，同时需要安装后端如Tensor low等

```
>>> pip install keras  
>>> pip install tensorflow
```

- Tensorflow分为GPU与CPU版本
 - 默认安装CPU版本
 - 如果计算机支持GPU，可安装GPU版本，提高神经网络的训练速度

序贯 (Sequential) 模型

- Sequential模型特点
 - 简单的线性模型
 - 由多个神经网络层按输入输出顺序线性堆叠而成
 - 只具有一个输出
- 函数式 (Functional) 模型在序贯模型基础上, 允许用户定义多输出、非循环有向或具有共享层的结构
- 基于Keras训练神经网络步骤
 1. 定义神经网络的结构, 说明组成网络的层类型、参数
 2. 定义神经网络的损失函数、优化器、性能评估指标, 然后编译模型
 3. 使用数据集训练模型、预测、性能分析

Keras神经网络常用层

- Dense层：全连接层，结点与下一层结点完全连接
- Activation层：激活层，对上一层的输出施加激活函数。
 - 常用的激活函数有softmax、relu、tanh、sigmoid等
- Dropout层：中断层
 - 在训练过程中，每次更新参数时按照一定概率，随机断开指定百分比（p）的输入神经元连接，用于防止过拟合。

Keras模型常用函数 (1)

- 编译模型

```
model.compile(loss, optimizer, metrics,...)
```

参数说明:

- loss: 损失函数，神经网络输出值与真实值之间的误差度量方法，有mean_squared_error、hinge、categorical_crossentropy等。
- optimizer: 优化器，神经网络的参数学习算法，有'SGD'、'RMSprop'、'Adagrad'、'Adam'等。
- metrics: 列表，给出所需的性能评估指标，如'accuracy'。

Keras模型常用函数 (2)

- 训练模型

`model.fit(x,y, batch_size, epochs, verbos,
validation_split, validation_data,...)`

参数说明:

x : 训练数据的输入Numpy数组。

y : 训练数据的目标（输出）Numpy数组。

batch_size : 整数，网络梯度更新的样本数，缺省值为32。

epochs : 整数，训练模型的迭代次数。

verbose : 日志显示方式，0：不输出，1：显示进度条，
2：每个epoch输出一行。

validation_split : 0~1， 用作验证集的训练数据比例，验证集不参与训练。

validation_data : 元组 (x_val, y_val)， 指定验证集。

Keras模型常用函数 (3)

- 评估模型

`model.evaluate(x_test,y_test,verbose,...)`

- 返回模型指定的误差loss和评估标准值metric

- 评估预测

`model.predict(x_test,verbose,...)`

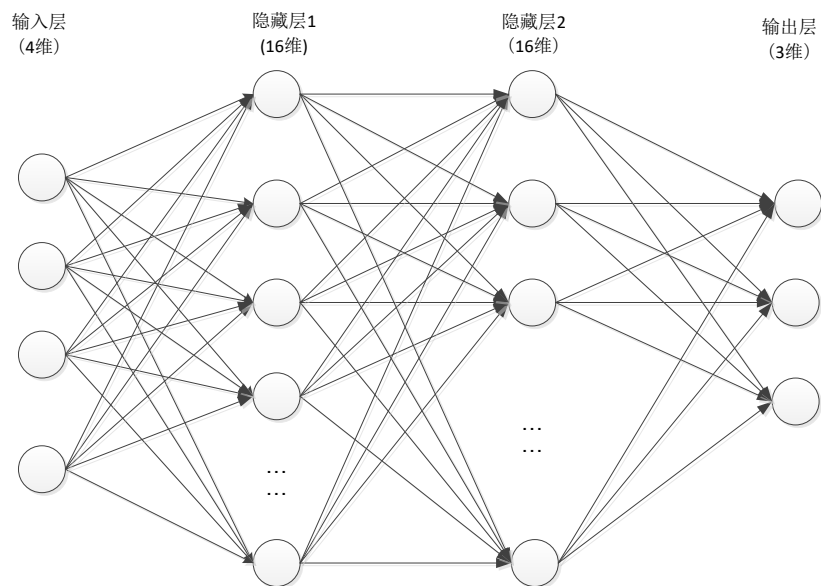
- 分类问题，返回样本属于每个类的预测概率

`model.predict_classes(x_test,verbose,...)`

- 仅用于序贯模型，为样本预测类别，输出类别标签值

例7-1：构建神经网络，为鸢尾花数据集训练分类器模型

- 鸢尾花数据集每个数据包含4个数据特征项，属于3种类型。
- 神经网络输入层维度为4，输出层维度为3
- 中间构建两个16结点隐层
- 隐层采用的激活函数为relu



- 堆叠神经网络层构建神经网络模型，编译

```
from keras.models import Sequential
from keras.layers import Dense, Activation

# 定义模型结构
model = Sequential()
model.add(Dense(units=16, input_dim=4))
model.add(Activation('relu'))
model.add(Dense(16))
model.add(Activation('relu'))
model.add(Dense(3))
model.add(Activation('softmax'))
#定义损失函数和优化器，并编译
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=["accuracy"])
```

- 用第5章例5-7预处理后的数据向量生成训练集和测试集

```
#预处理代码见例5-7，此处省略
train_x, test_x, train_y, test_y = train_test_split(X, y,
train_size=0.8, test_size=0.2, random_state=0)
#keras多分类问题需要将类型转化为二元类矩阵
from keras.utils import np_utils
train_y_ohe = np_utils.to_categorical(train_y, 3)
test_y_ohe = np_utils.to_categorical(test_y, 3)
```

- 用训练集训练模型，用测试集评估模型性能

```
model.fit(train_x, train_y_ohe, epochs=50, batch_size=1, verbose=2,
          validation_data=(test_x, test_y_ohe))
# 评估模型
loss, accuracy = model.evaluate(test_x, test_y_ohe, verbose=2)
print('loss = {}, accuracy = {}'.format(loss, accuracy) )

prob = model.predict(x_test, verbose=2)
classes = model.predict_classes(x_test, verbose=2)
print('测试样本数：', len(classes))
print("分类概率:\n", prob)
print("分类结果:\n", classes)
```

```
#keras不直接提供分类性能报告，可使用sklearn.metrics提供的功能
from sklearn.metrics import classification_report
print( classification_report(y_test, classes) )
```

- 将预测样本输入分类器，输出显示构建的分类器在鸢尾花测试集的30个样本上都预测正确。

综合练习

1. 葡萄酒数据集 (wine.data) 搜集了法国不同产区葡萄酒的化学指标。试建立决策树、SVM和神经网络3种分类器模型，比较各种分类器在此数据集上的效果。

【提示】：每种分类器，需要对参数进行尝试，找出此种分类算法的较优模型，再与其他分类器性能进行比较。

2. 基于Keras建立深度神经网络，为葡萄酒数据集训练分类器，比较不同规模神经网络模型的参数个数，模型的训练时间，分类性能。